National Teachers College

629 J. Nepomuceno St., Quiapo, Manila

School of Arts, Sciences and Technology

Final Project/Exam: C# Application Development for Sustainable Development

(**PesoPinoy**: Microfinance & Loan Management System)

ITELEC 2 | IT Elective 2

Custorio, April Nicole

Dela Cruz, Junelle F.

Maningo, Gabriel C.

Sabiniano, Francis Jr. B.

Santos, Maureen C.

3.2 BSIT

Prof. Justin Louise R. Neypes

March 2026

PesoPinoy: Microfinance & Loan Management System

(SDG 1 – No Poverty)

I. INTRODUCTION

1.1 Project Overview

PesoPinoy is a desktop-based Community Microfinance Loan Management System built on the NET / C# platform using an N-Tier Architecture. The system is designed to empower small-scale lending cooperatives and microfinance institutions (MFIs) to efficiently manage the complete lifecycle of micro-loans, borrower accounts, savings, insurance, and risk analysis through a single, cohesive platform.

In the Philippines, many individuals rely on small loans to address financial needs but are unable to access traditional bank loans due to strict requirements such as collateral, credit history, and income verification. As a result, alternative lending services have become a common financial option for many borrowers.

Despite their important role in providing financial support, many small-scale lending institutions still rely on manual record-keeping. This can lead to errors in records, delayed payment tracking, and limited financial monitoring. PesoPinoy addresses these challenges by providing a secure and organized digital system that improves loan tracking. borrower management, and financial reporting. Through better financial management and responsible lending practices, the system also supports **Sustainable Development Goal 1 (No Poverty**) by helping strengthen community-based lending institutions.

1.2 Statement of the Problem

Many small lending groups and cooperatives still rely on manual systems such as spreadsheets or handwritten ledgers. These methods often result in:

- Incorrect loan balance computation
- Missed due dates
- Poor borrower risk assessment
- Data redundancy and inconsistency
- Lack of financial performance analytics

There is a need for a secure and automated system that:

1. Accurately computes loan interest and amortization schedules.
2. Tracks borrower payments and remaining balances.
3. Evaluates borrower financial risk.
4. Provides graphical summaries of financial data.
5. Maintains reliable and persistent database storage.

II. REQUIREMENT & ANALYSIS

2.1 Functional Requirements

ID	Module	Description
FR-01	**Admin Authentation**	The system shall provide a secure login form with SHA-256 hashed password validation and session management. The system shall also allow administrators to securely log out by invalidating the current session.
FR-02	**Borrower Management**	The system shall allow administrators to add new borrowers with complete profile details, update existing borrower records, and display full borrower profiles. The system shall also support the deletion of borrower records.
FR-03	**Loan Management**	The system shall allow the creation of new loans with configurable principal, interest rate, and loan term. The system shall automatically compute either compound or simple interest and generate a complete amortization schedule upon loan creation. Additionally, the system shall track and display the remaining loan balance and classify the loan status as Active, Pending, Partial, or Completed.
FR-04	**Payment Management**	The system shall record loan payments and automatically update the remaining balance of each loan. The system shall also calculate and apply penalties for late or missed payments and maintain a complete payment history log for every borrower and loan.
FR-05	**Savings Management**	The system shall support deposit and withdrawal transactions against a borrower's savings account and automatically update savings balances.
FR-06	**Insurance**	The system shall allow registration of micro-insurance policies per borrower with defined premium and coverage amounts. The system shall support activation and deactivation of insurance coverage per borrower.
FR-07	**Risk Scoring**	The system shall compute a risk score for each borrower using their monthly income, employment status, age, presence of a guarantor, and payment history. Based on the calculated score, borrowers shall be classified into Low Risk, Medium Risk, High Risk, or Very High Risk categories.
FR-08	**Dashboard**	The system shall display total of borrowers, number of active loans, savings total, overdue payments, as well as monthly loan collections and risk distribution as visual charts.
FR-09	**JSON Backup**	The system shall allow export of all system data to a JSON file and restoration from a JSON file asynchronously.
FR-10	**Logging**	The system shall log all exceptions to an external file with timestamps and error details.

2.2 Non-Functional Requirements

A. Performance

- Loan amortization schedule generation shall complete within 2 seconds for loan terms up to 60 months.
- Risk score computation for a single borrower shall complete within 500 milliseconds.
- JSON backup export shall process up to 10,000 records within 5 seconds using asynchronous I/O.
- Dashboard chart rendering shall complete within 1.5 seconds of module load.

B. Security

- All administrator passwords shall be stored using SHA256 hashing — plaintext passwords must never be persisted.
- Session tokens shall be invalidated immediately upon logout.
- All database queries shall use parameterized statements to prevent SQL injection.
- All financial operations shall be restricted to authenticated administrators only.

C. Robustness & Reliability

- All external operations (file I/O, database) shall be wrapped in global try-catch with error logging.
- JSON restore shall validate schema integrity before applying changes to prevent data corruption.
- The system shall remain responsive during async backup/restore operations via Task-based async processing.
- Referential integrity shall be enforced at the Data Access Layer to prevent orphaned records.

2.3 Complex Analysis (Big-O Notation)

The core Business Logic method analyzed is the **Loan Amortization Schedule Generation Algorithm**, located in the Loan Processing component of the Business Logic Layer. This algorithm computes the payment schedule by calculating the interest, principal payment, and remaining balance for each payment period.

This section provides a detailed analysis of the algorithm's time and space complexity, along with an illustrative example of the computation over multiple periods.

No.	Algorithm Step	Complexity	Explanation
1	Initialize loan balance	O(1)	Assigns the initial loan amount to the balance variable.
2	Iterate through loan periods	O(n)	Loops through each payment period in the loan term.
3	Compute interest for current balance	O(1)	Calculates interest using the current balance and interest rate.
4	Compute principal payment	O(1)	Determines the principal portion of the payment by subtracting interest.
5	Update remaining balance	O(1)	Subtracts the principal portion from the current balance.
6	Store payment record	O(n)	Adds each computed payment record to the amortization schedule list.
7	Overall Complexity	O(n)	Linear complexity: total operations grow proportionally to the number of payment periods.

The algorithm performs calculations sequentially for each payment period. Each step inside the loop executes in constant time O(1), and the loop runs for n periods. Therefore, the total time complexity is O(n). Storing each payment record requires linear space relative to the number of periods, resulting in a space complexity of O(n) as well.

Example Iteration Table

This table illustrates the algorithm for a loan of ₱100,000 with an annual interest rate of 12%, showing sample computation for the first 6 months. The table can be extended to 12 or 24 months for full-term analysis.

Month	Payment	Interest (12%/12)	Principal	Remaining Balance
1	8,885	1,000	7,885	92,115
2	8,885	921	7,964	84,151
3	8,885	841	8,044	76,107
4	8,885	761	8,124	67,983
5	8,885	680	8,205	59,778
6	8,885	598	8,287	51,491
12	8,885	278	8,607	0 (fully paid)

Observation:

The number of operations and storage of payment records grow linearly with the number of periods (n). This visualizes the O(n) complexity both in computation and memory usage.

Complexity Summary

Complexity Type	Value	Explanation
Time Complexity	O(n)	Algorithm processes each of the n payment periods once, performing constant time calculations per period.
Space Complexity	O(n)	Stores n payment records in the amortization schedule, growing linearly with the number of periods.
Worst-case Scenario	O(n)	Maximum loan term length determines the total number of iterations.
Best-case Scenario	O(1)	For a loan with 1 period, only initialization and one iteration occur.
Average-case Scenario	O(n)	Typical loan term of multiple months: linear growth in both computation and storage.

The analysis of the Loan Amortization Algorithm demonstrates that it is efficient and scalable, with both time and space complexity growing linearly with the number of payment periods (O(n)). Each step of the algorithm executes in constant time (O(1)), and storage requirements increase predictably as records are added for each month. This ensures that the system can handle loans of varying lengths without performance issues. The visual example and flowchart further support the linear nature of the computation, confirming that the algorithm is well-optimized for practical use in the Business Logic Layer.

III. DESIGN & SPECIFICATION

3.1 N-Tier Architecture Diagram

PesoPinoy follows a strict 3-tier N-Tier Architecture separating presentation, business logic, and data access concerns.

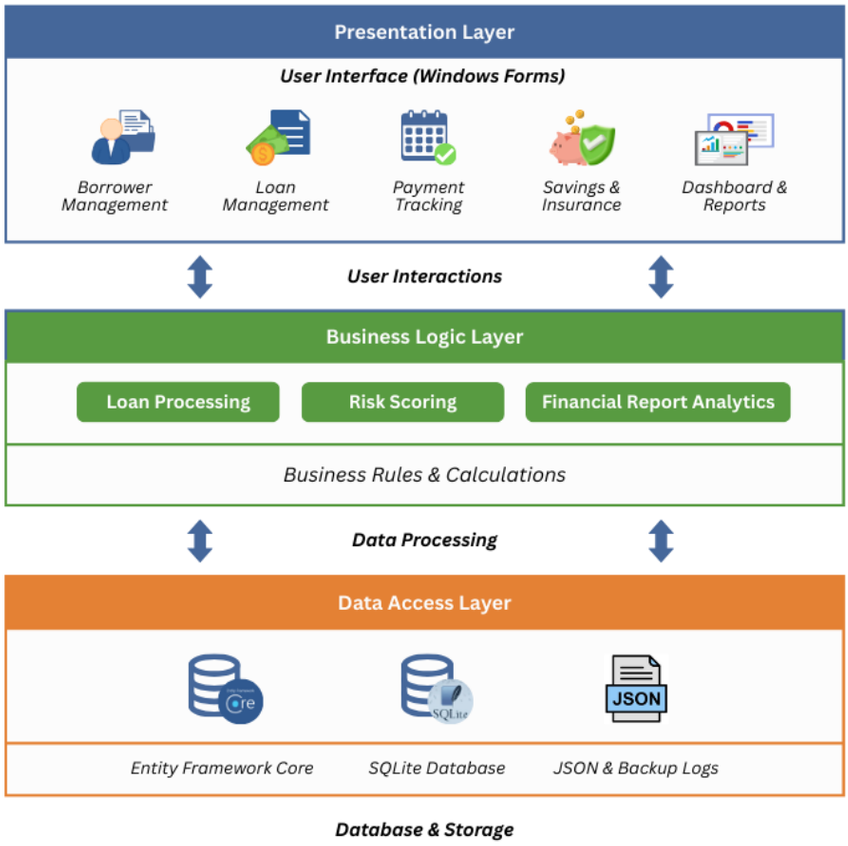


Figure 1. *N-Tier Architecture Diagram*

Figure 1 illustrates the **N-Tier Architecture diagram** of the PesoPinoy Microfinance and Loan Management System, organized into three layers: Presentation, Business Logic, and Data Access. This separation improves maintainability, scalability, and system organization.

The **Presentation Layer** is the user interface where administrators manage borrower records, process loans, track payments, handle savings and insurance, and view reports. It collects input and displays processed information.

The **Business Logic Layer** is the system's core, handling financial computations and business rules such as loan processing, risk scoring, and report analytics. It calculates loan interest, updates payment balances, and evaluates borrower risk, keeping business rules separate from the UI and database.

The **Data Access Layer** manages communication with data storage using Entity Framework Core for CRUD operations. Data is stored in an SQLite database, with JSON files for backup and logging.

This N-Tier Architecture ensures the system operates modularly, efficiently managing financial data, supporting lending operations, and maintaining reliable data storage.

3.2 Entity Relationship Diagram (ERD)

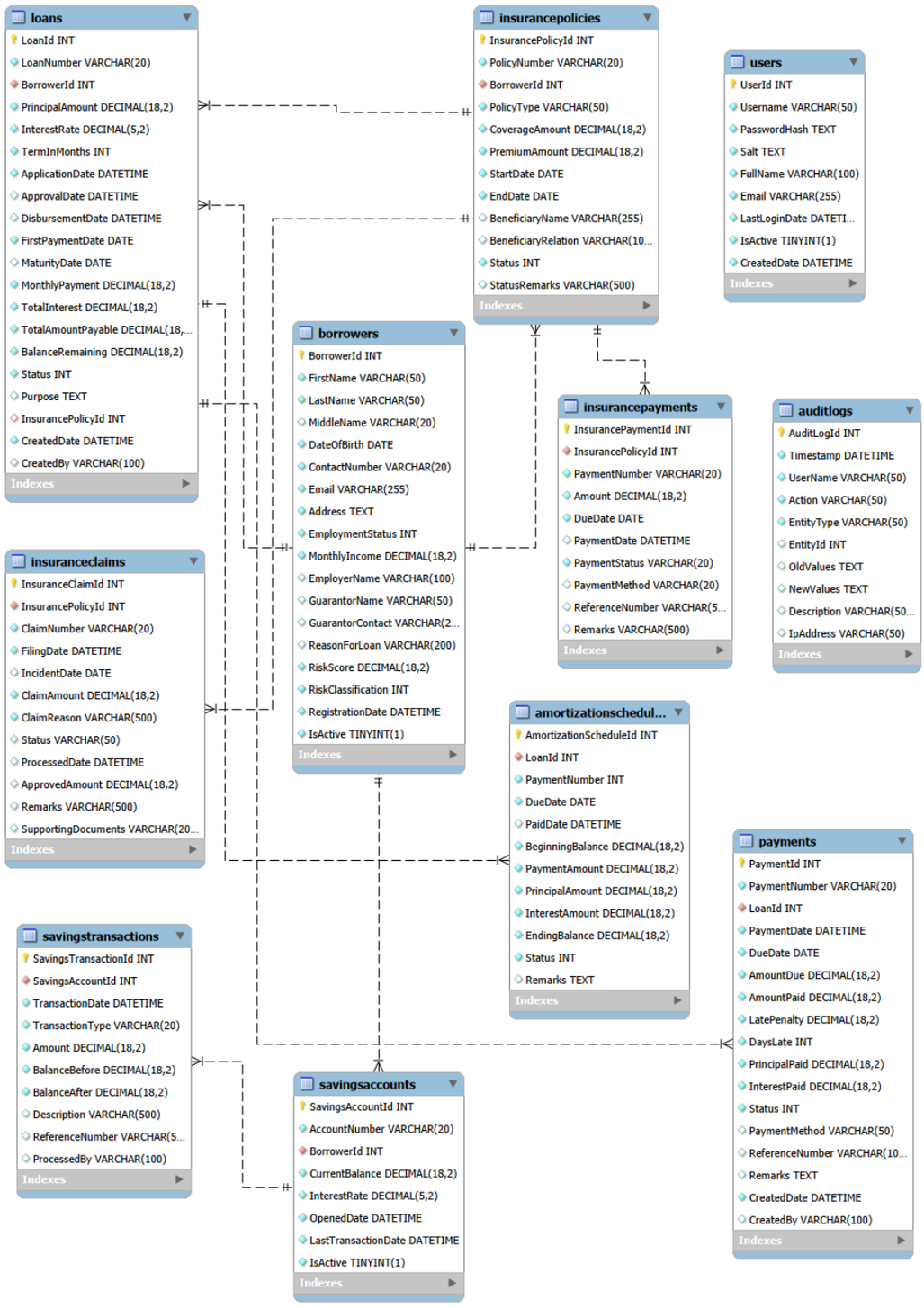


Figure 2. *Entity Relationship Diagram (ERD)*

Figure 2 shows the **Entity Relationship Diagram (ERD)** for PesoPinoy which shows a complete microfinance and loan management system designed to track borrowers, loans, payments, savings, and insurance. The main entities include borrowers, loans, payments, savings accounts, insurance policies, and users, each serving a specific purpose in the system.

The **Borrowers** entity stores all client information, including personal details, contact information, employment status, monthly income, and risk classification. This serves as the central hub for all client-related data in the system.

The **Loans** entity manages all loan details, including principal amount, interest rate, term, payment schedules, and loan status. Each loan is linked to a specific borrower, and can optionally be connected to an insurance policy for loan protection. The loan status tracks whether a loan is pending, active, paid, or defaulted.

The **Payments** entity records all loan payments made by borrowers, tracking payment dates, amounts paid, late penalties, and how much goes toward principal versus interest. This helps maintain accurate records of borrower payment history and remaining balances.

The **Amortization Schedules** entity breaks down each loan into individual payment periods, showing the breakdown of principal and interest for each scheduled payment. This helps borrowers see their payment schedule and helps staff track upcoming payments.

The **Savings Accounts** and **Savings Transactions** entities manage borrower savings. Each borrower can have a savings account that tracks deposits, withdrawals, and current balance. All transactions are recorded to maintain an audit trail.

The **Insurance Policies** and **Insurance Claims** entities handle the insurance side of the system. Borrowers can have insurance policies linked to their loans, with coverage amounts and premium payments tracked. When needed, claims can be filed against these policies. The **insurance payments** entity tracks premium payments made for each policy.

The **Users** entity stores system user information, including login credentials and access levels, allowing the system to control who can access different features. The **Audit Logs** entity records all significant actions in the system, maintaining a complete history of who did what and when for security and compliance purposes.

All in all, the relationships show how the system works: borrowers can have multiple loans, savings accounts, and insurance policies. Each loan generates multiple payments and an amortization schedule. Insurance policies can have multiple claims and premium payments. All significant actions are recorded in audit logs. The database uses foreign key links to make sure the data stays consistent and follows the business rules for lending, payments, savings, and insurance. This design helps both staff and clients by organizing all microfinance operations in a simple, efficient way for the PesoPinoy platform, supporting the goal of financial inclusion and poverty reduction.

3.3 Use Case Diagram & Data Flow Description

Figure 3. *Use Case Diagram*

3.3.1 Use Case Summary

Actor	Use Case	Description
Administrator	Login Auth	Actor enters credentials; system validates against SHA256 hash in DB.
Administrator	Manage Borrowers	CRUD operations on borrower profiles; view full profile summary.
Administrator	Create Loan	Admin inputs loan details; system computes interest and generates amortization schedule.
Administrator	Record Payment	Admin records payment; system updates balance and applies penalties.
Administrator	Manage Savings	Admin deposits or withdraws from borrower's savings account.
Administrator	Manage Insurance	Admin registers, activates, or deactivates borrower insurance policies.
Administrator	View Risk Score	Admin triggers risk computation; system returns classification.
Administrator	View Dashboard	Admin views charts of collections, loan status, and risk distribution.
Administrator	Backup & Restore	Admin exports/imports data as JSON file; handled asynchronously.

3.3.2 Data Flow Diagram (DFD - Level 0)

The Level 0 DFD, also called the Context Diagram, represents the entire PesoPinoy system as a single unified process (Process 0). It establishes the system boundary and shows all external entities that interact with the system along with the high-level data flows crossing that boundary. No internal processes are decomposed at this level.

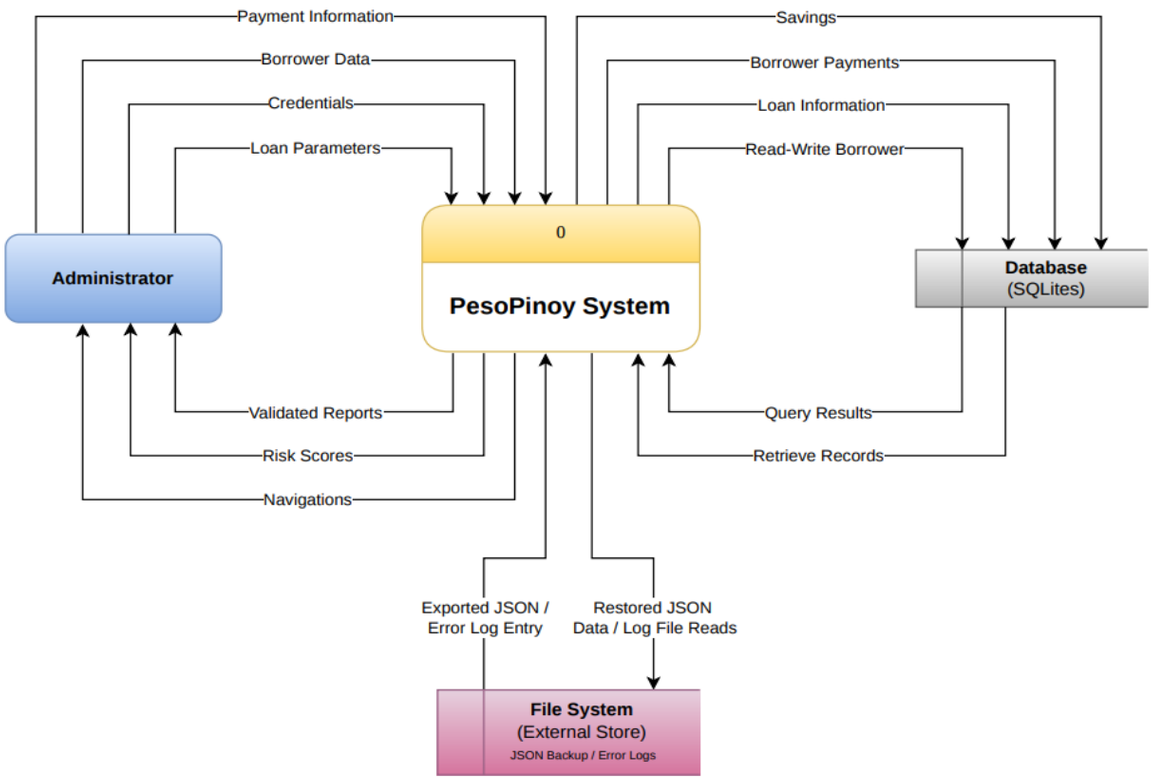


Figure 4. *Data Flow Diagram (Level 0)*

DFD – Level 0 : External Entities

Entity	Role & Description
Administrator	The sole human actor of the system. The Administrator initiates all operations — logging in, managing borrowers, creating loans, recording payments, managing savings and insurance, triggering risk scoring, viewing dashboards, and issuing backup/restore commands.
Database (SQLite)	The persistent relational data store (SQLite). Holds all master and transactional records: borrowers, loans, payments, savings, insurance policies, risk scores, and admin credentials. The system reads from and writes to the database as part of every core operation.
File System (External Store)	An external file repository used for two purposes: (1) storing JSON backup snapshots exported by the Backup & Restore module, and (2) holding the plain-text error log file written by the Logging & Exception module.

3.3.3 Data Flow Diagram (DFD - Level 1)

The Level 1 DFD decomposes Process 0 from the Context Diagram into ten individual sub-processes (1.0 through 10.0).

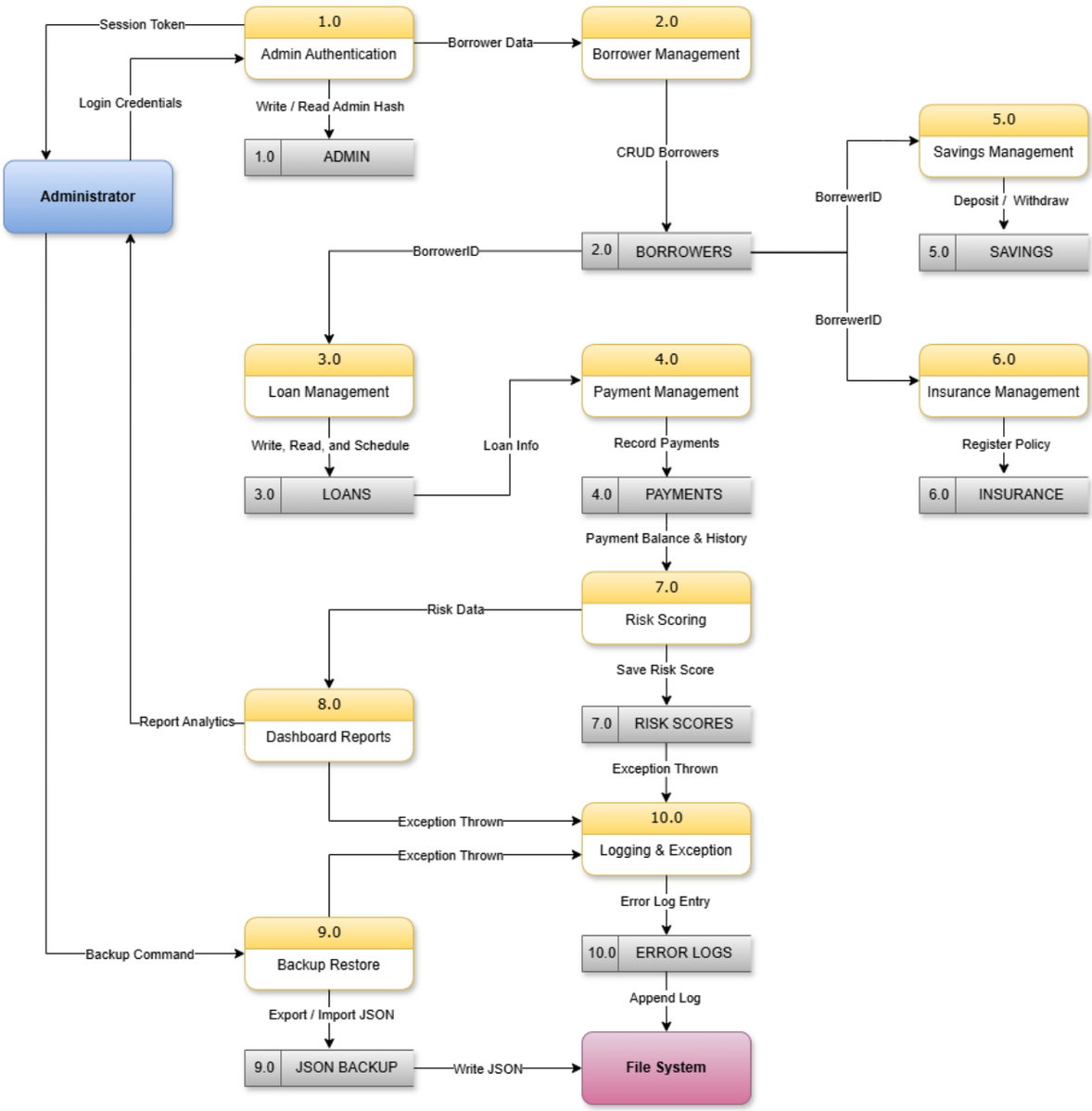


Figure 5. *Data Flow Diagram (Level 1)*

DFD – Level 1: Inter-Process Data Flows

From	To	Data Flow Label & Description
P1.0 Admin Auth	P2.0 Borrower Mgmt	**Borrower Data.** After successful login, the session context passes borrower-related access to the Borrower Management module to begin CRUD operations.
P2.0 Borrower Mgmt	P3.0 Loan Mgmt	**BorrowerID.** The unique identifier of a borrower is passed to Loan Management so new loans can be linked to the correct borrower record via foreign key.
P2.0 Borrower Mgmt	P5.0 Savings Mgmt	**BorrowerID.** Passed to Savings Management to associate deposit and withdrawal transactions with the correct borrower's savings account.
P2.0 Borrower Mgmt	P6.0 Insurance Mgmt	**BorrowerID.** Passed to Insurance Management so new micro-insurance policies are correctly linked to the intended borrower.
P3.0 Loan Mgmt	P4.0 Payment Mgmt	**Loan Info.** Active loan details (loan ID, outstanding balance, schedule) passed to Payment Management for use in payment validation, balance updates, and penalty computation.
P4.0 Payment Mgmt	P7.0 Risk Scoring	**Payment Balance & History.** Aggregated payment records including on-time flags and penalty counts, forwarded to the Risk Scoring module for computing the Payment Punctuality Score component.
P7.0 Risk Scoring	P8.0 Dashboard	**Risk Data.** Computed borrower risk classifications and scores forwarded to the Dashboard Reports module for rendering the Borrower Risk Distribution chart and portfolio analytics.
P7.0 Risk Scoring	P10.0 Logging	**Exception Thrown.** Any unhandled exception occurring during risk score computation is immediately forwarded to the global Logging & Exception handler.
P8.0 Dashboard	P10.0 Logging	**Exception Thrown.** Any data-access or rendering error in the Dashboard Reports module is forwarded to the Logging & Exception handler for recording.
P9.0 Backup Restore	File System	**Write JSON.** Serialized JSON backup snapshots are asynchronously written to the external File System during backup operations.
P10.0 Logging	File System	**Append Log.** Formatted error log entries are appended to the external plain-text error log file in the File System, independent of the database write.

3.4 Algorithm Flowchart Description

The Loan Amortization Algorithm Flowchart illustrates how the PesoPinoy system processes and generates a loan repayment schedule. The process begins with administrator authentication and loan data input, followed by iterative calculations of monthly interest, principal, and remaining balance until the loan term is completed. After generating the amortization schedule, the administrator may return to the dashboard or log out of the system.

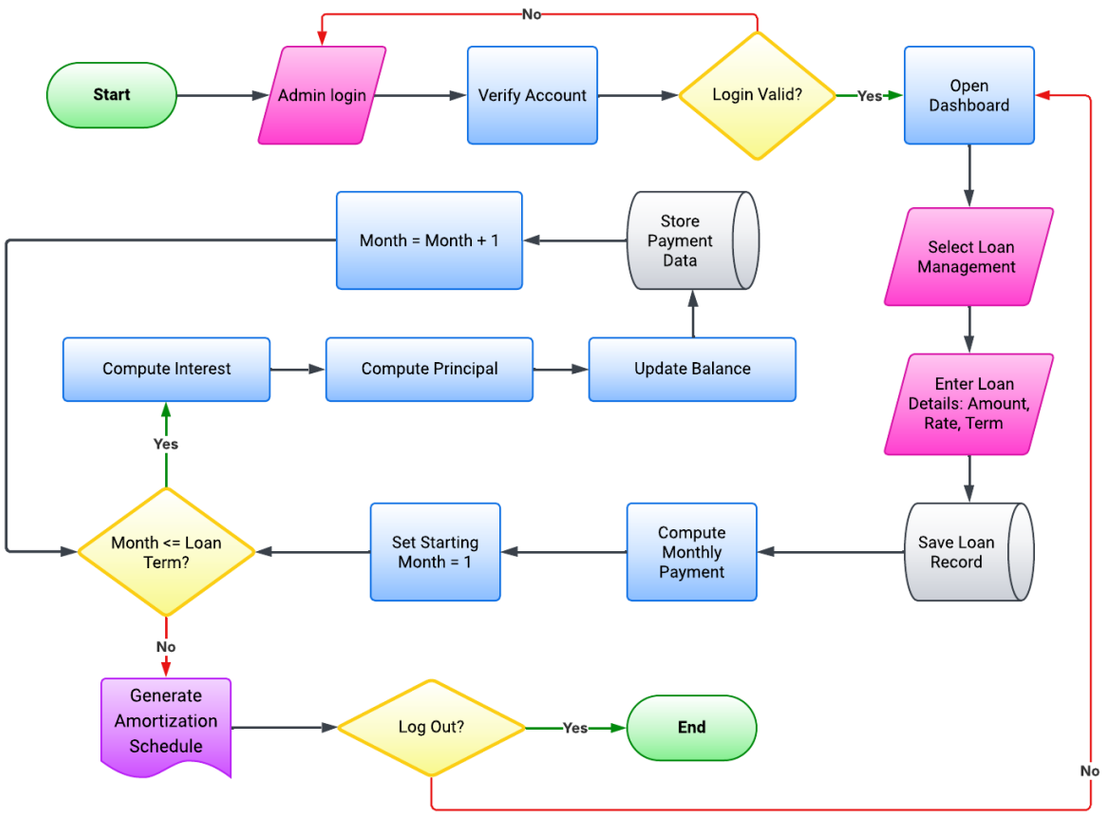


Figure 6. *Loan Amortization Flowchart*

#	Algorithm Step	Description
1	Start System	The system begins operation and prepares for administrator authentication.
2	Administrator Login	The administrator enters credentials to access the system.
3	Verify Account	The system checks the credentials against stored records.
4	Open Dashboard	The administrator is directed to the system dashboard.
5	Select Loan Management	The administrator selects the loan management module.
6	Enter Loan Details	Loan amount, interest rate, and loan term are entered into the system.

#	Algorithm Step	Description
7	Save Loan Record	The system stores the loan information in the database.
8	Compute Monthly Payment	The system calculates the fixed monthly payment.
9	Initialize Month Counter	The system sets the starting month to 1.
10	Compute Interest	The system calculates the interest portion for the current period.
11	Compute Principal	The system determines the principal portion of the payment.
12	Update Balance	The system updates the remaining loan balance.
13	Store Payment Record	Payment details are saved in the database.
14	Increment Month	The system moves to the next payment period.
15	Generate Amortization Schedule	The system generates the final loan repayment schedule.
16	Logout	The administrator may end the system session.

IV. INDIVIDUAL CONTRIBUTIONS

The table below details the specific layer, implemented by each team member:

#	Layer / Module	Assigned Member
1	Presentation Layer.	Custorio, April Nicole
		Dela Cruz, Junelle F.
		Santos, Maureen C.
2	Business Logic Layer	Custorio, April Nicole
		Santos, Maureen C.
		Maningo, Gabriel C.
3	Data Access Layer	Sabiniano, Francis Jr.
		Dela Cruz, Junelle F.
4	SDAD Documentation	Custorio, April Nicole
5	SDAD Diagrams	All Members

V. APPENDICES

5.1 PesoPinoy System Flowchart

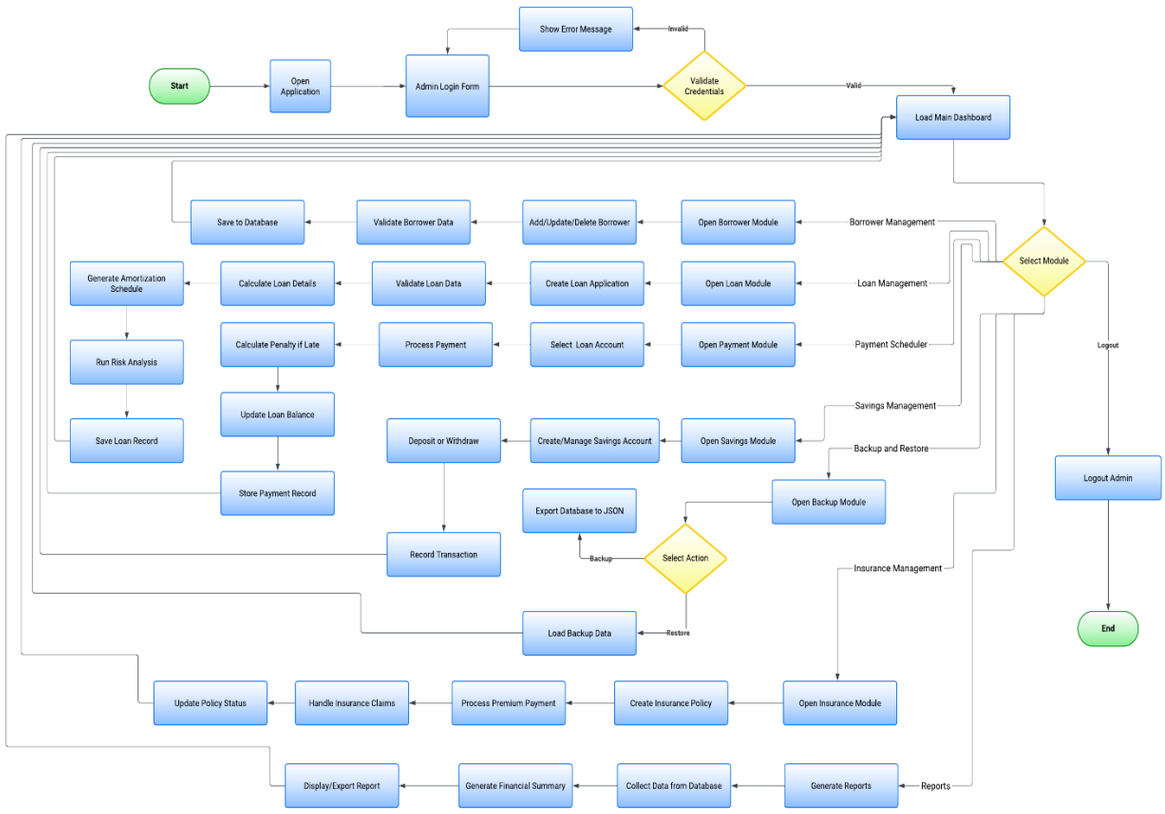


Figure 7. *Administrator Login Screen*

Figure 7 shows the **PesoPinoy System Flowchart** which illustrates the complete operational workflow of the PesoPinoy Community Microfinance Loan Management System. It shows how the administrator interacts with the system, starting from authentication, navigating different modules, processing borrower and loan transactions, and generating reports. The flowchart demonstrates how the system integrates multiple financial management functions such as borrower management, loan processing, payments, savings, insurance, and database backup within a single platform.

5.2 System Features Description

The following table describes each module's features as implemented in PesoPinoy:

1. Administrator Authentication Module
Secure Login Form.
Input fields for username and password with masked password entry.
SHA256 Hashed Password Storage.
Passwords are never stored in plaintext; SHA256 hash is compared against DB value.
Credential Validation.
Invalid credentials display a user-friendly error message without revealing which field is wrong.
Logout Function.
Clears session state and redirects to the login screen.

2. Dashboard & Data Visualization Module
Monthly Loan Collections.
Bar chart showing total collections per month for the current year.
Active Loans.
Number of active loans.
Borrower Risk Distribution.
Stacked bar or pie chart showing count of Low, Medium, High, or Very High risk borrowers.

3. Borrower Management Module
Add New Borrower.
Form captures full name, contact number, address, and monthly income.
Edit/Update Borrower Details.
Pre-populated edit form allows updating any borrower field.
Delete Borrower Record.
Soft/hard delete with referential integrity check; warns if active loans exist.
View Borrower Profile.
Summary view displaying borrower details, active loans, savings balance, insurance status, and risk classification.

4. Loan Management Module
Create New Loan.
Accepts principal, annual interest rate, and term in months.
Auto Interest Computation.
Computes monthly interest using the reducing-balance method.
Automatic Amortization Schedule Generation.
Full schedule generated at loan creation showing due dates, principal, interest, and balance per period.
Remaining Balance Calculation.
Updated in real-time as payments are recorded.
Loan Status Tracking.
Status is automatically updated to Active, Completed, or Defaulted based on payment behavior.

5. Payment Management Module
Record Payment.
Admin selects loan and enters payment amount; system validates against schedule.
Auto Update Remaining Balance.
Immediately reduces loan balance upon payment confirmation.
Automated Penalty Computation.
System checks payment date against due date; applies configured penalty rate for late payments.
Payment History Viewing.
Chronological list of all payments for a given loan with penalty breakdown.

6. Savings Management Module
Deposit Savings.
Admin records deposits to borrower's savings account.
Withdraw Savings.
Validates sufficient balance before processing withdrawal.

7. Insurance Module
Register Insurance Policy.
Creates policy record with defined premium (monthly payment) and coverage amount (maximum claimable).
Insurance Claims Filing.
File an insurance claim by selecting an existing policy and providing the following details: Policy Selection, Incident Date, Claim Amount, Claim Reason.
Process Insurance Claim.
Administrators review submitted insurance claims and process them by either **approving or rejecting** the filed claim.
Premium Payment Management.
Input Borrower’s amount of their insurance premiums and display the payment details.

8. Reports and Analytics Module
Reports and Analytics Module.
The system shall display key summary metrics including Total Borrowers, Active Loans, Total Collections, and Average Risk Score
Loan Performance Report.
Generate a visual chart that shows the number of loans based on their current status to help administrators monitor loan performance.
Automatic Risk Score Classification.
Display a graphical representation of borrower risk categories to help analyze the distribution of low, medium, high, and very high risk borrowers.
Monthly Collections Report.
Generate a report summarizing loan repayments and collections within a specific month.
Borrower Demographics Report.
Analytics related to borrower distribution to help administrators understand borrower characteristics and patterns.

9. JSON Backup & Restore Module
Export Database to JSON.
Serializes all database tables to a structured JSON file for backup.
Restore System Data from JSON.
Deserializes JSON and reloads data into the database with schema validation.
Async Processing.
Uses C# async/await (Task-based) to prevent UI freezing during long I/O operations.

10. Logging & Exception Handling Module
Global Try-Catch Implementation.
Application-level exception handler wraps all service method calls.
Error Logging to External File.
Exceptions are written to a persistent log file (e.g., app_errors.log) outside the database.
Log Timestamp and Error Details.
Each log entry includes: DateTime, error message, stack trace, and originating module name.

5.3 Application Screenshots

The following screenshots show the actual running PesoPinoy system, captured from the live application:

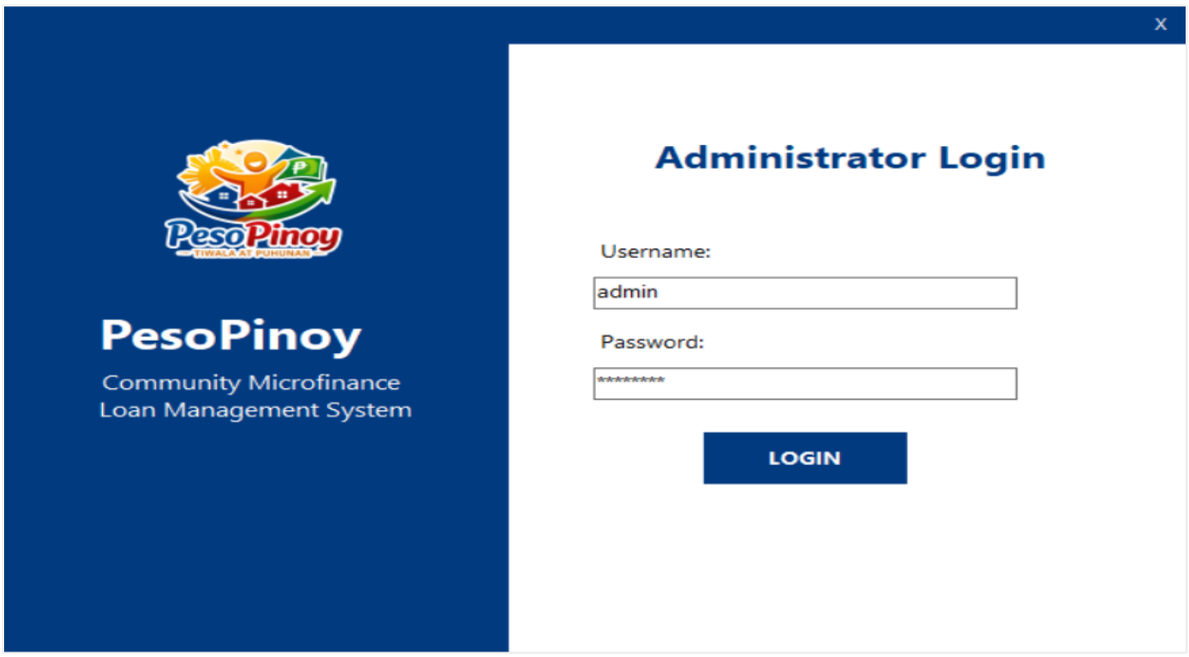


Figure 8. *Administrator Login Screen*

Figure 8 shows the PesoPinoy login form with username and password fields for secure administrator access.

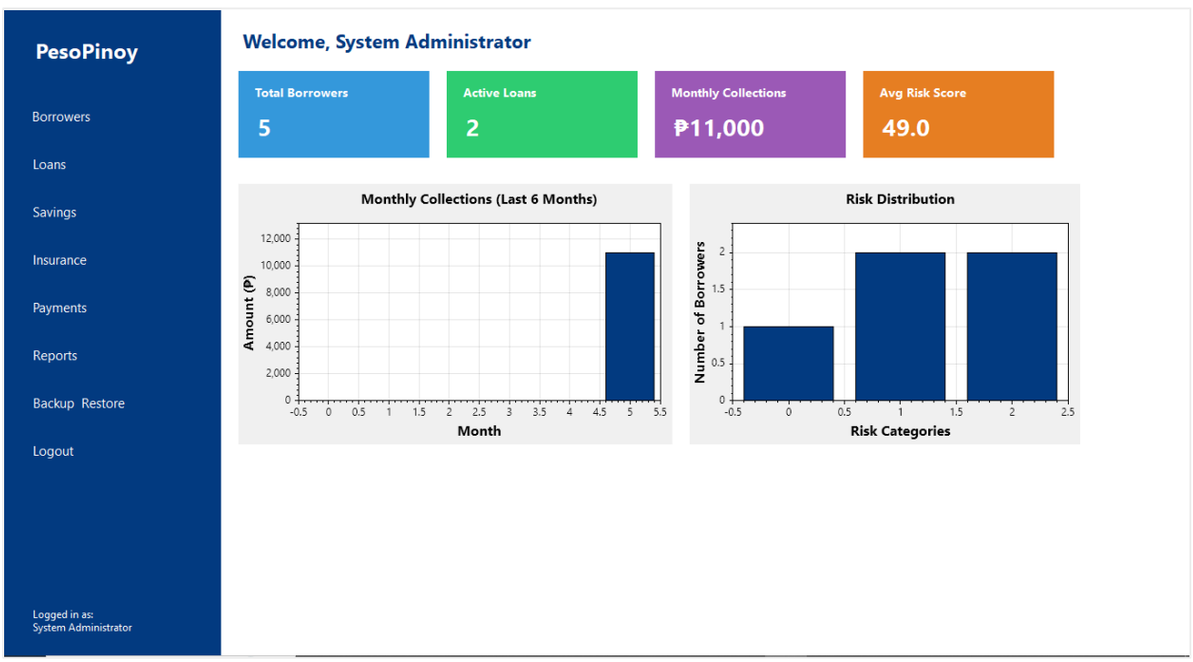


Figure 9. *Administrator Dashboard*

Figure 9 shows the Administrator Dashboard displaying key metrics: Total Borrowers, Active Loans, Total Savings, and Overdue Payments, along with Monthly Collections and Risk Distribution charts.

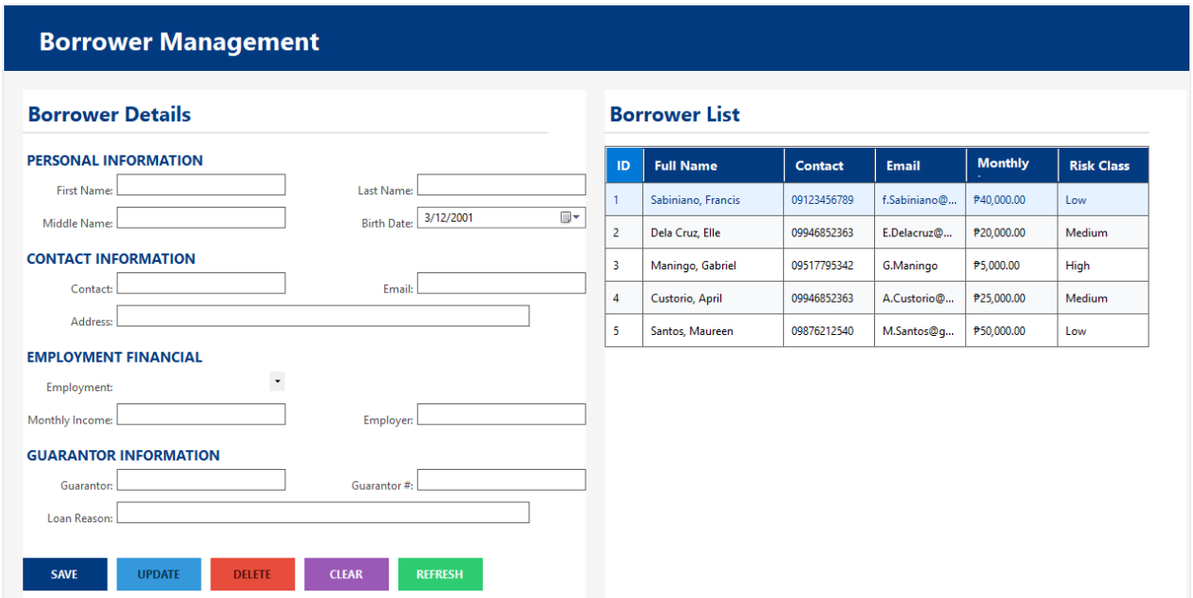


Figure 10. *Borrower Management*

Figure 10 shows the Borrower Management module with the borrower list and the full data entry form including fields for First/Last Name, Birth Date, Contact, Email, Employment, Monthly Income, Guarantor, and Loan Reason.

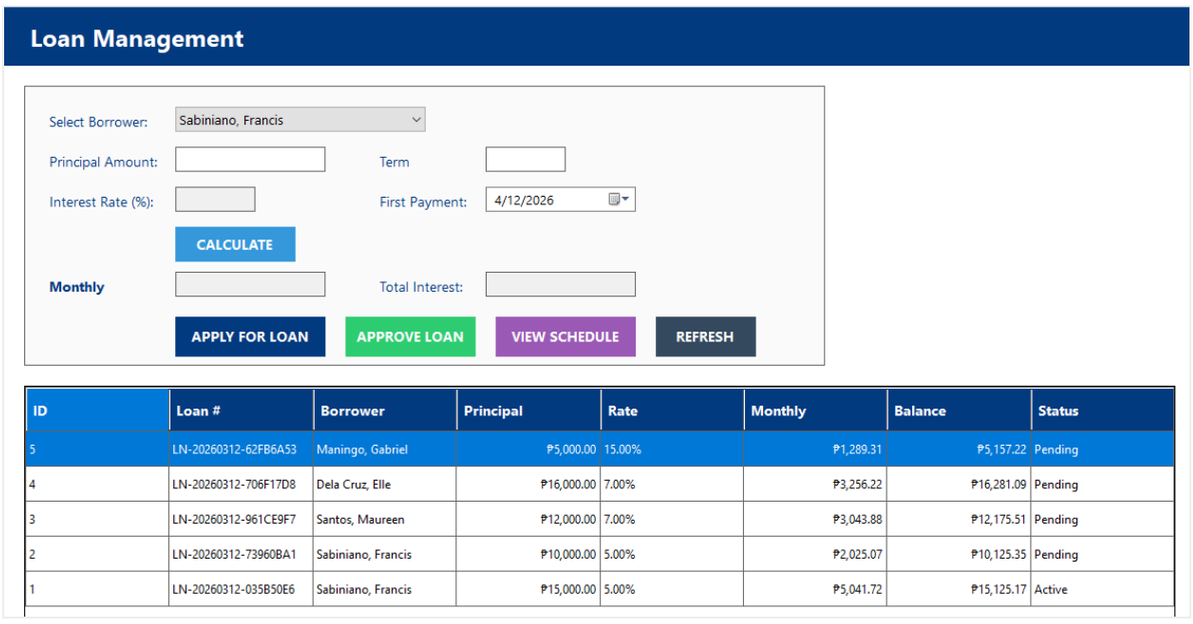
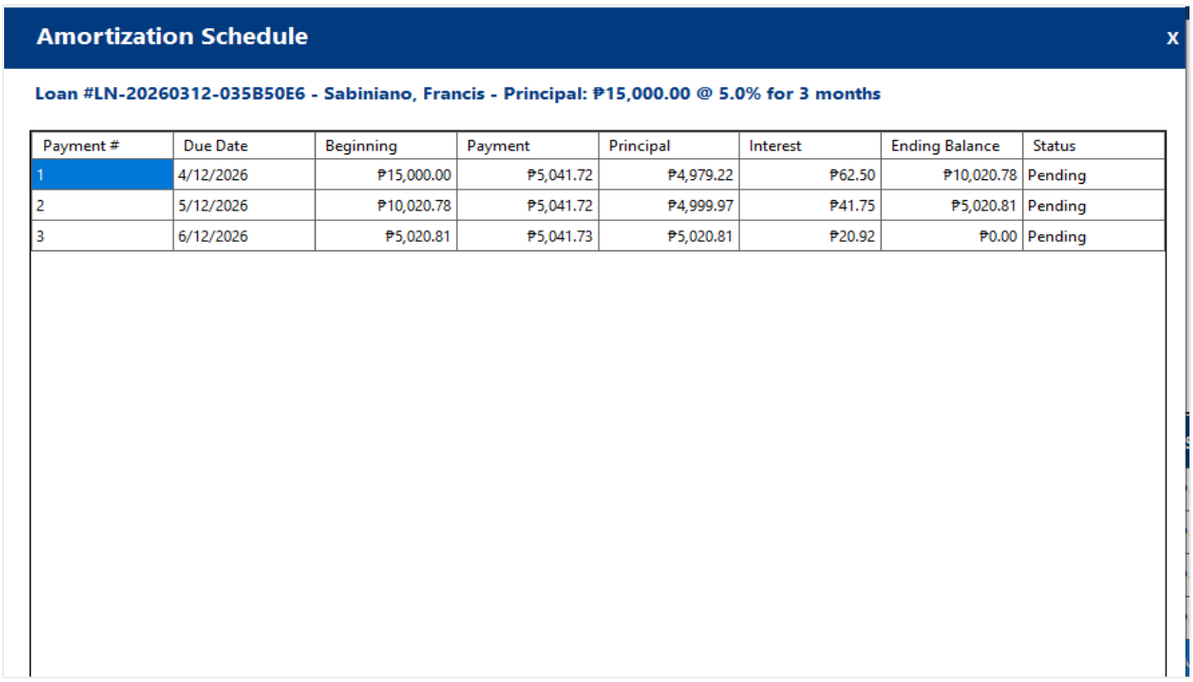


Figure 11. *Loan Management*

Figure 11 shows the Loan Management screen where administrators can select a borrower, input principal amount, interest rate, and term, then Calculate and Apply for Loan. Active loans are listed with Loan #, Borrower, Principal, Rate, Monthly payment, Balance, and Status.



Payment #	Due Date	Beginning	Payment	Principal	Interest	Ending Balance	Status
1	4/12/2026	₱15,000.00	₱5,041.72	₱4,979.22	₱62.50	₱10,020.78	Pending
2	5/12/2026	₱10,020.78	₱5,041.72	₱4,999.97	₱41.75	₱5,020.81	Pending
3	6/12/2026	₱5,020.81	₱5,041.73	₱5,020.81	₱20.92	₱0.00	Pending

Figure 12. *Amortization Schedule Viewer*

Figure 12 shows the Amortization Schedule dialog window launched from Loan Management. The schedule table displays columns for Payment #, Due Date, Beginning Balance, Payment Amount, Principal portion, Interest portion, Ending Balance, and Status — generated automatically upon loan creation.

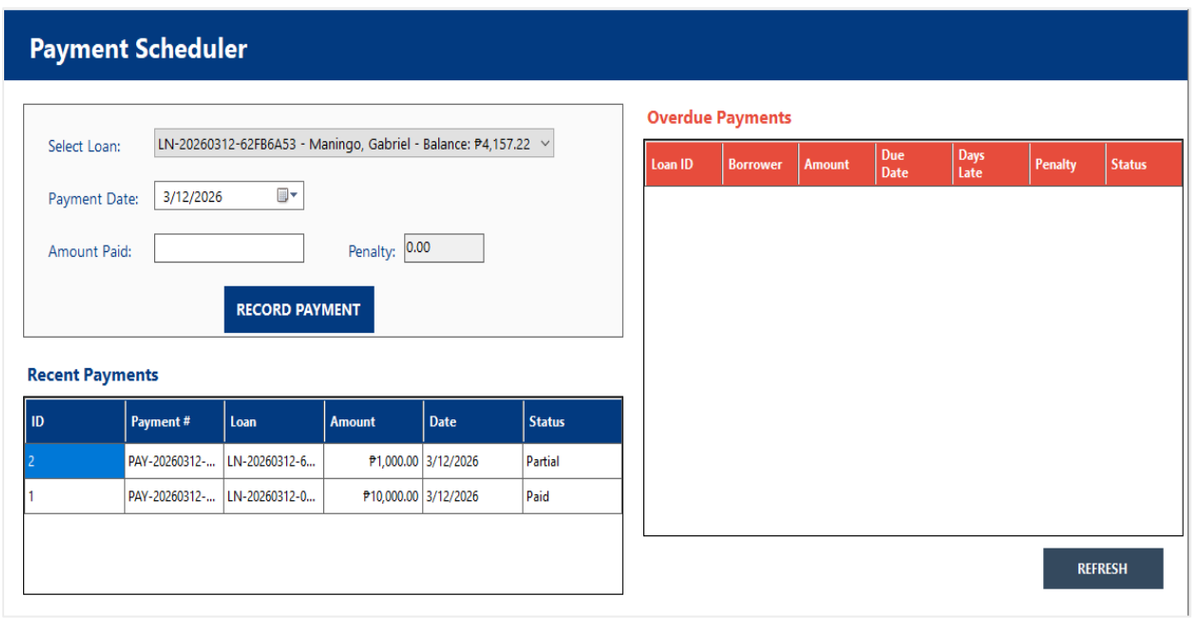


Figure 13. *Payment Scheduler*

Figure 13 shows the Payment Scheduler module where administrator record payments against a selected loan, with automatic penalty computation for overdue accounts.

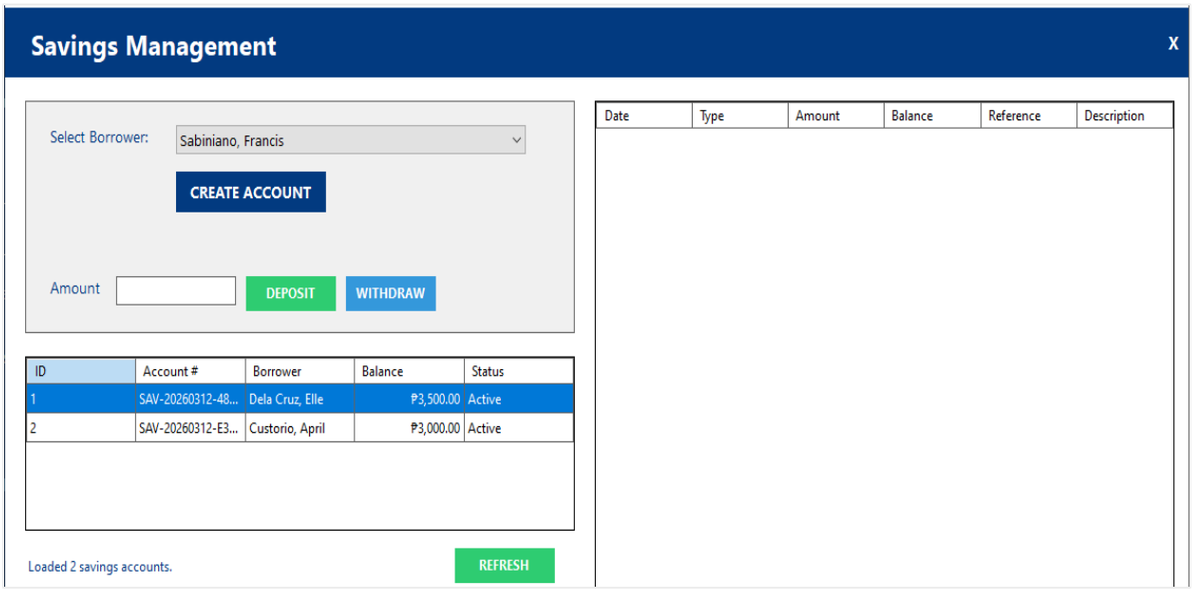


Figure 14. *Savings Management*

Figure 14 shows the Savings Management module displaying borrower savings accounts, account balances, and a transaction history panel showing Date, Type, Amount, Balance, Reference, and Description per entry.

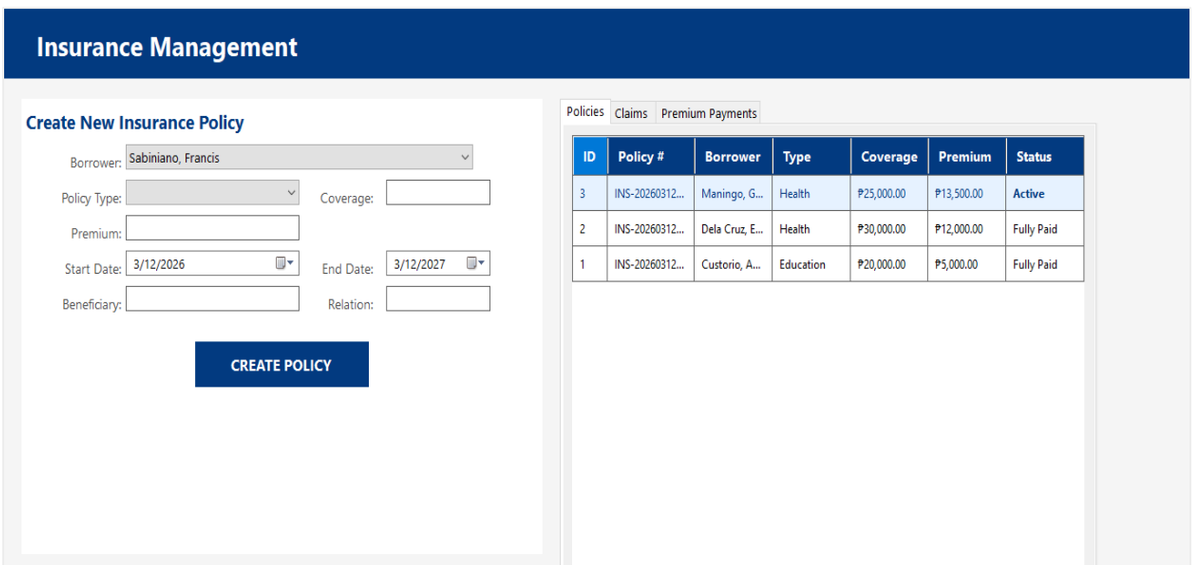
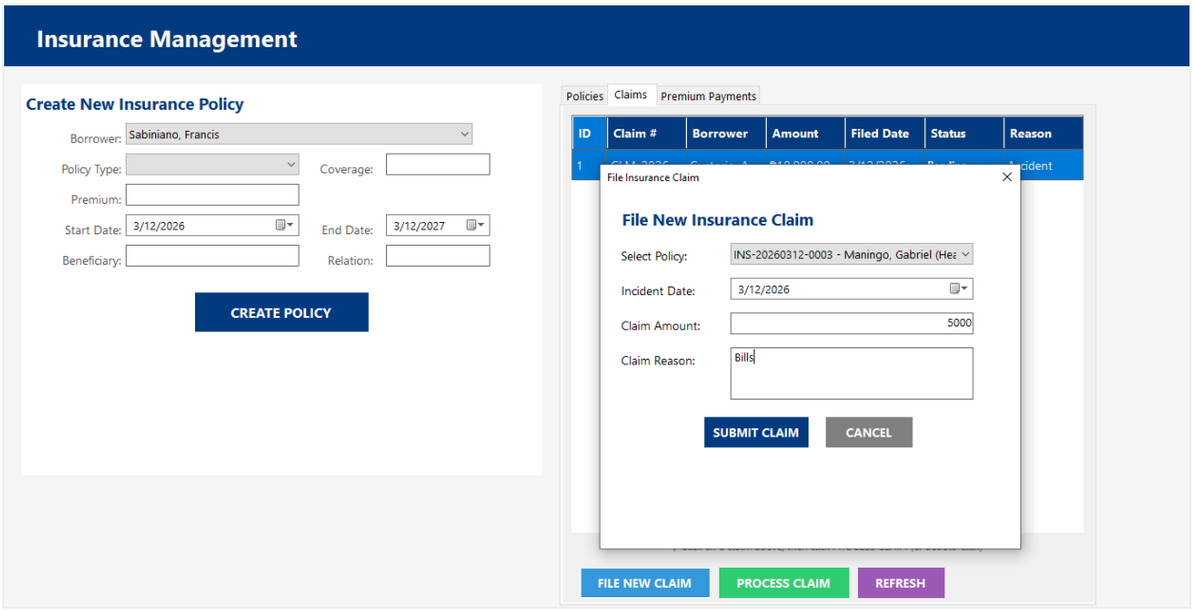


Figure 15. *Insurance Management*

Figure 15 shows the Insurance Management which enables the creation and tracking of insurance policies. Admin can input borrower details, policy type, coverage, premium, and beneficiary information. A table displays all active policies with key details such as policy number, borrower, type, and premium.



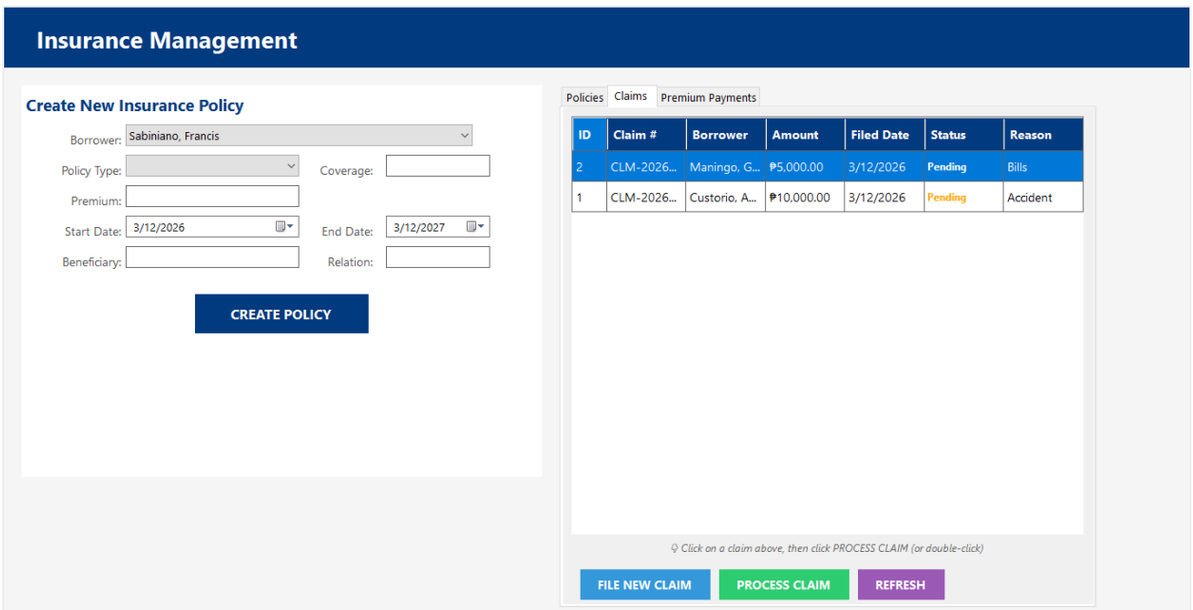


Figure 16. *Insurance Management (Claims)*

Figure 16 shows the Insurance Management Claims Tab that supports filing and monitoring insurance claims. Admin can select a policy, specify incident details, claim amount, and reason. Submitted claims are listed in a table showing claim number, borrower, amount, date filed, status, and reason.

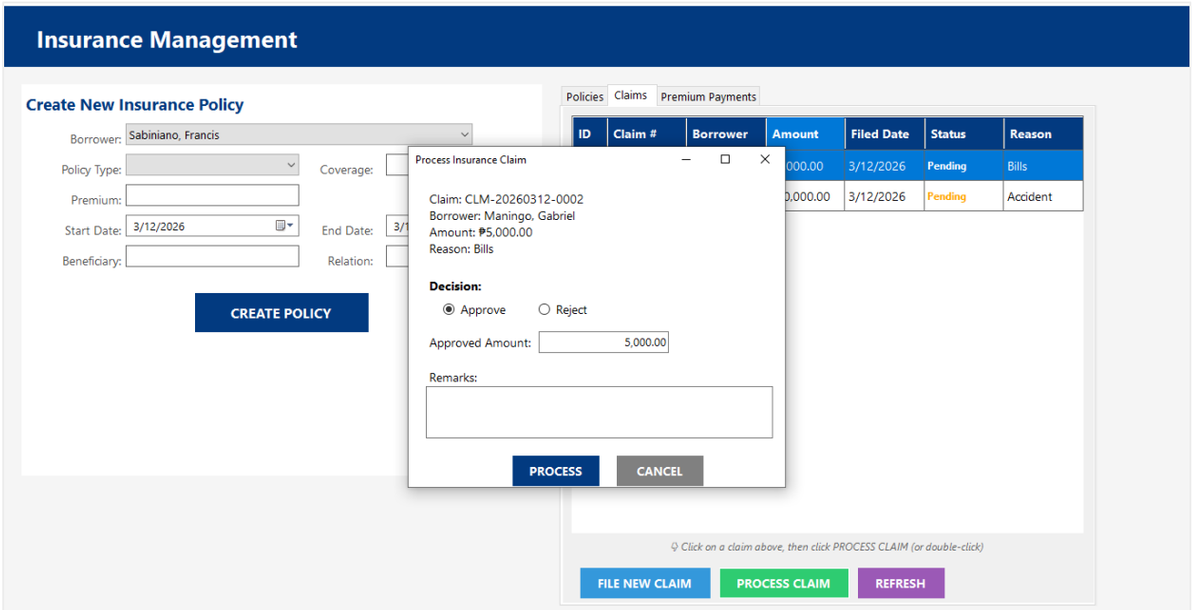


Figure 17. *Process Insurance Claim*

Figure 17 shows Process Insurance Claim where administrator review and act on submitted claims. Each claim includes borrower details, reason, and requested amount. Admin can approve or reject claims, specify the approved amount, and add remarks before finalizing the decision.

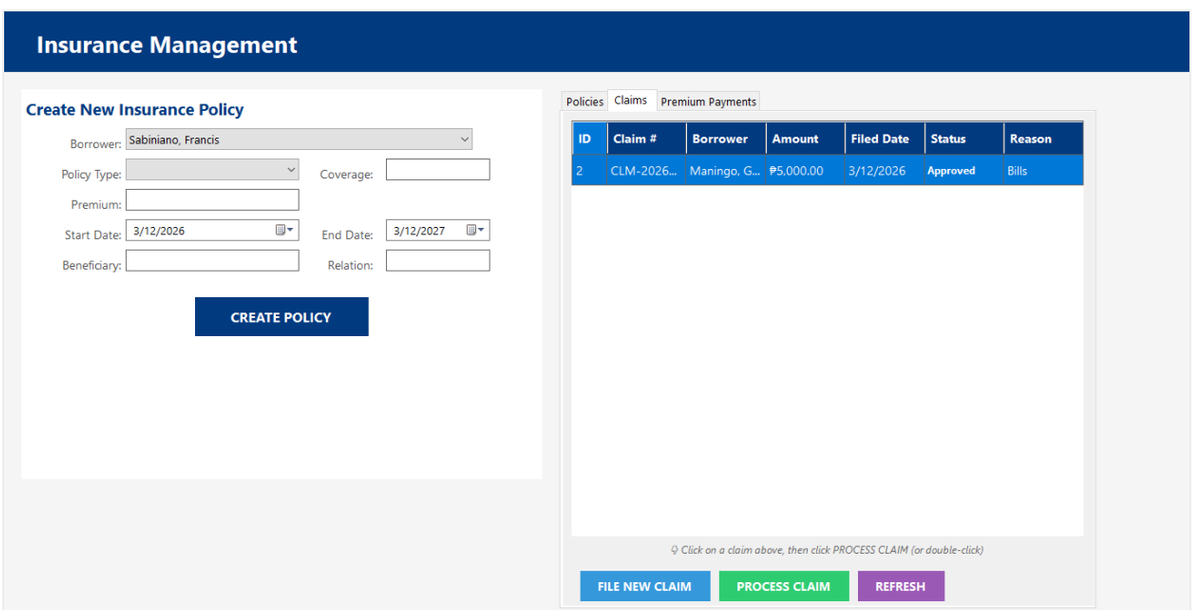


Figure 18. *Approved Process Insurance Claim*

Figure 18 shows the Approved Process Insurance Claim that display the claim have already been processed and approved. It provides a summary of claim details such as claim ID, borrower, amount, filed date, status, and reason, ensuring transparency and easy tracking of finalized claims.

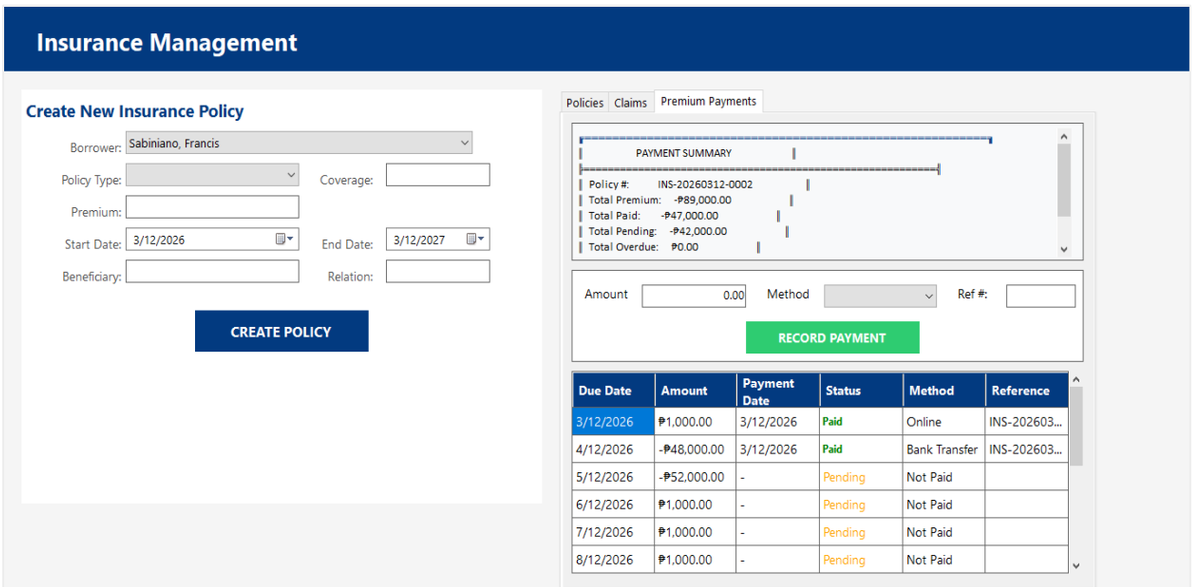


Figure 19. *Insurance Management (Premium Payments)*

Figure 19 shows Insurance Management (Premium Payments) where administrator manages insurance premium collections. It shows a payment summary with total premium, paid amount, balance, and overdue status. Admin can record payments by entering the amount, method, and reference number. A detailed table lists due dates, payment status, and methods for each installment.

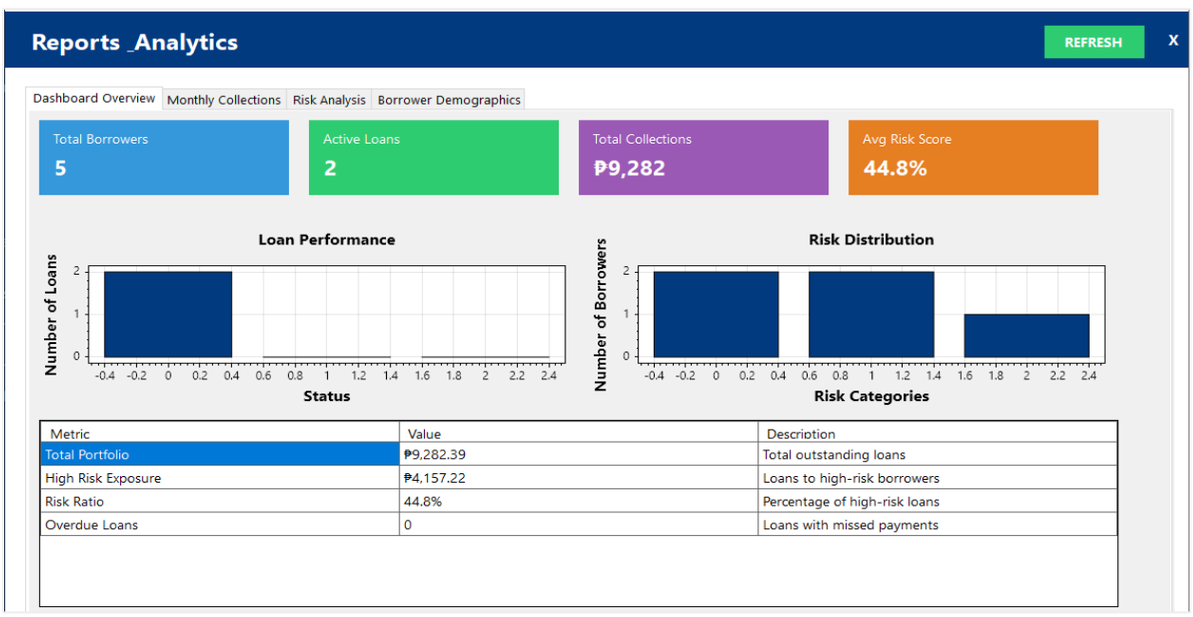


Figure 20. *Reports Analytics (Dashboard Overview)*

Figure 20 shows the Reports Analytics (Dashboard Overview), providing an overview of key system data. It displays visual charts and summaries, such as loan collections, borrower statistics, and risk distribution, to help administrators quickly assess the system's performance.

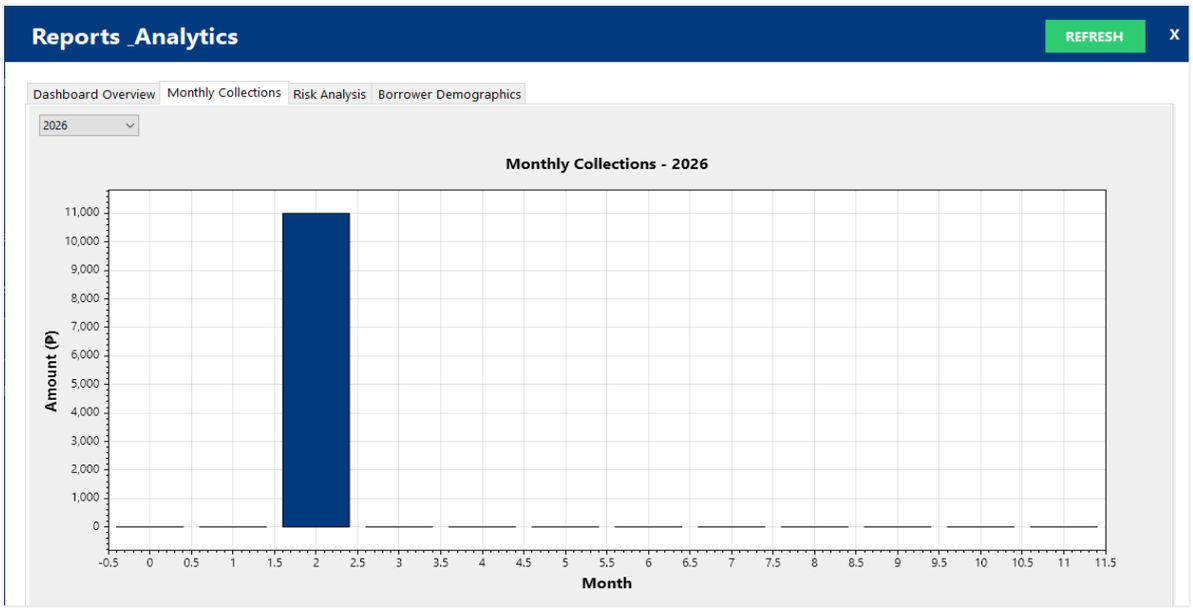


Figure 21. *Reports Analytics (Monthly Collections)*

Figure 21 shows the Reports Analytics (Monthly Collections), highlighting monthly collections along with borrower statistics and risk distribution. It provides visual charts that help administrators quickly track financial performance and overall system status.

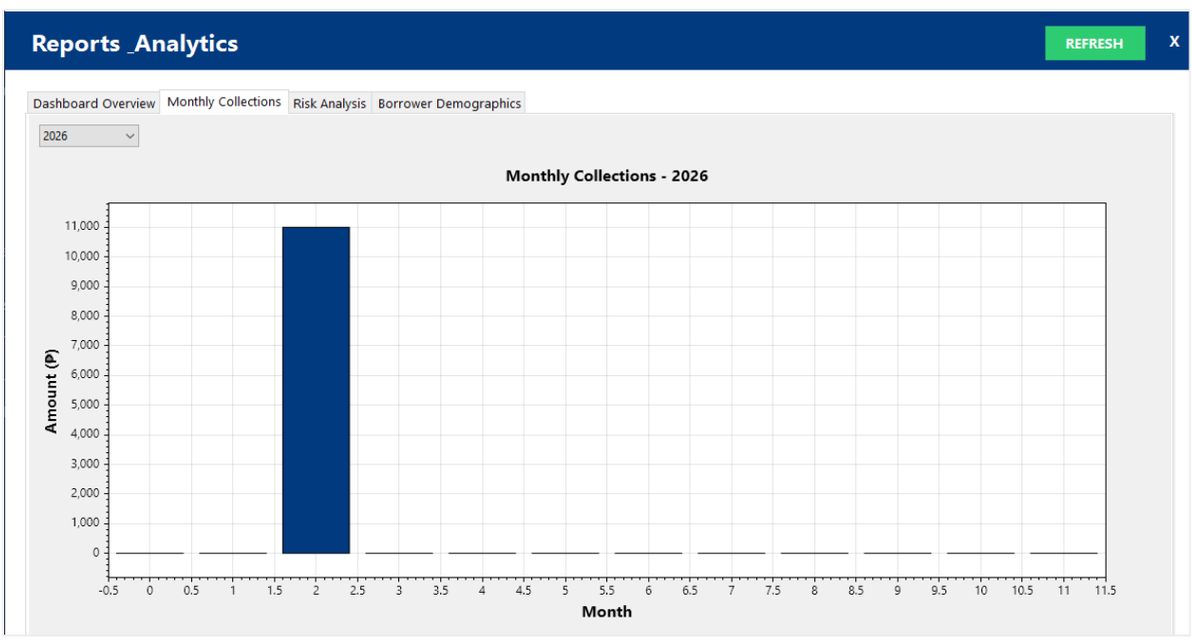


Figure 22. *Reports Analytics (Risk Analysis)*

Figure 22 shows the Risk Analysis module, which evaluates borrowers based on their monthly income, employment status, age, presence of a guarantor, and payment history. The system automatically classifies borrowers as Low, Medium, High, or Very High risk, helping administrators make informed lending decisions.

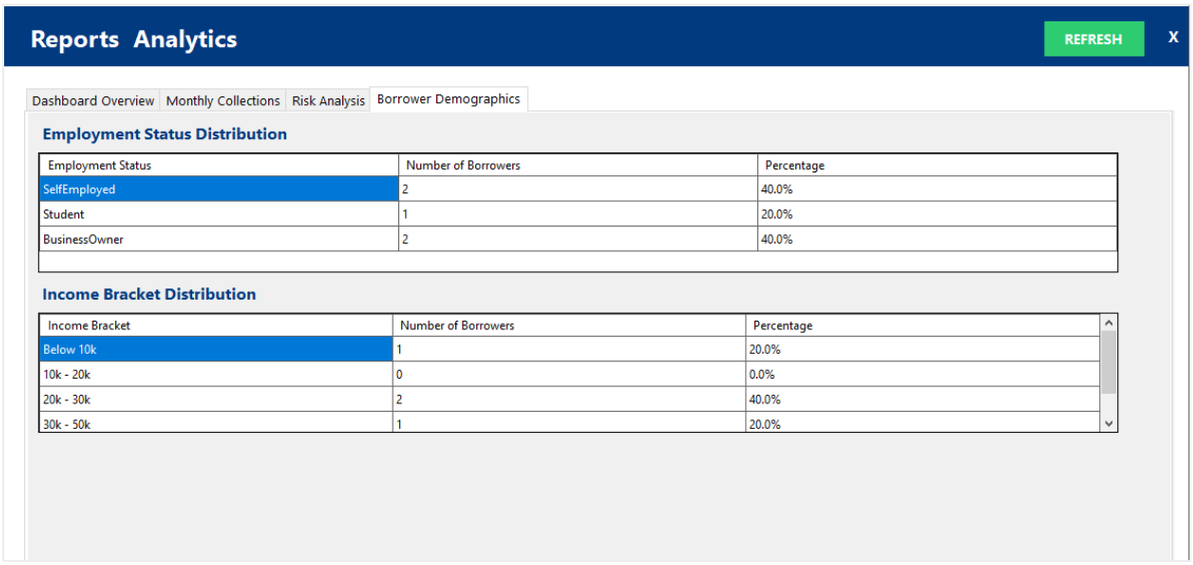


Figure 23. *Reports Analytics (Borrower Demographics)*

Figure 23 shows the Borrower Demographics module, displaying employment status and income bracket distributions with borrower counts and percentages. It helps administrators understand the profile of their borrowers.

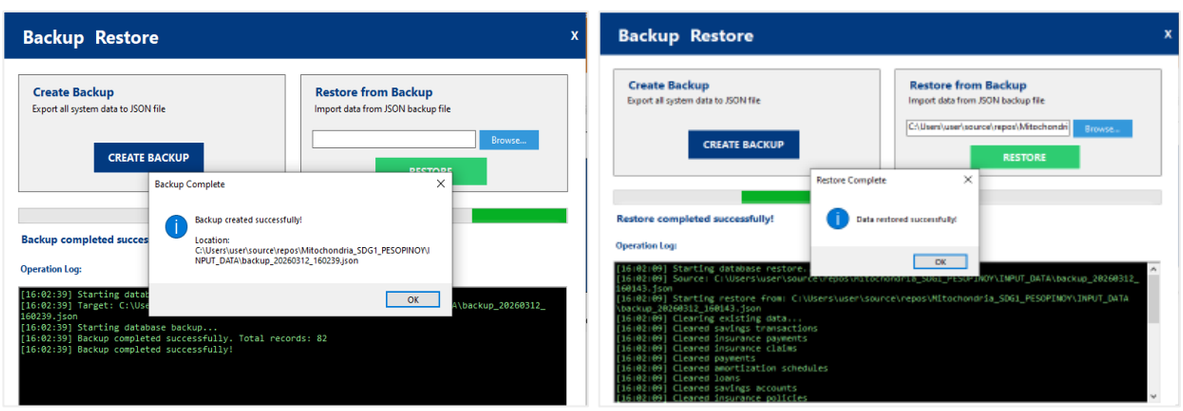


Figure 24. *Backup and Restore*

Figure 24 shows the Backup and Restore module, which allows administrators to export system data to a JSON file and restore data from a backup. It also includes an operation log to track all backup and restore activities.

VI. TOOLS AND REFERENCES

- **DeepSeek AI.** Used as a debugging and guidance tool during the development process. It helped the developers identify possible errors in the code and provided explanations for programming concepts, allowing the students to better understand and implement the logic independently.

- **Lucidchart and Canva.** Used to create system diagrams such as system architecture, use case diagrams, and workflow visualizations for the project documentation.

- **draw.io.** Used to construct technical diagrams including system structure and process flow representations.

- Layaoen, C. W. G., & Takahashi, K. (2022). *Can microfinance lending crowd out informal lenders? Evidence from the Philippines*. *Journal of International Development, 34*(2), 379–414. https://doi.org/10.1002/jid.3604 According to Takahashi, microfinance lending in the Philippines can reduce reliance on informal lenders and help households access credit more formally, which supports financial inclusion. The study highlights the importance of tailored microfinance schemes to reach underserved populations.

- WeSolve Foundation. (2025). *Microfinance and household shocks in the Philippines*. https://www.wesolve.ph/philippine-microfinance-report. Microfinance programs in the Philippines play a crucial role in improving financial resilience for low-income households, allowing them to better cope with unexpected shocks and manage their economic needs.

- Binaluyo, J. P., Agustin, N. B., & Santos, A. R. (2025). *Microfinance institutions and services: Consumer perspectives and the path to standardized guidelines*. *Risk Governance & Control: Financial Markets & Institutions, 15*(2), 80–91. https://doi.org/10.22495/rgcv15i2p7. Research on microfinance institutions in the Philippines shows that MFIs provide essential financial services and face challenges that, when addressed, can enhance financial inclusion and support poverty alleviation.